

Iterator and ListIterator

Iterator offers us a mechanism to traverse a Collection. To cycle through the elements of a collection you can use an object that implements Iterator or ListIterator interfaces.

Iterator enables you to traverse a collection **obtaining** or **removing** elements.

First step: getting the iterator.

```
ArrayList<Integer> l = new ArrayList<Integer>(); // I use an ArrayList as an example of collection
for (int i=1; i<=10; i++){ // I add numbers from 1 to 10 to the ArrayList
    l.add(i);
}
Iterator it = l.iterator(); // I get the iterator from the list.
```

Then I can use the iterator to cycle through the elements of the ArrayList (or another collection).

// ArrayList with all the initial elements

```
while (it.hasNext()) { // I have to check if I have another element in the collection
    Integer num = it.next(); // I get each element of the collection
    System.out.print(num + "\t");
}
// 1    2    3    4    5    6    7    8    9    10
```

I can remove an element from the collection:

```
while (it.hasNext()) {
    if(it.next()%2==1) // if the element is odd I remove it
        it.remove();
}
System.out.println(l); // ArrayList after removing odd numbers
// 2    4    6    8    10
```

ListIterator is a bidirectional iterator that traverses list in the forward and in the backward direction.

In ListIterator you have all the methods of Iterator plus other methods to update and add elements to the list. You can also get the index of the next element in the list.

First, I have to get the ListIterator object:

```
LinkedList<Integer> l = new LinkedList<Integer>(); // I use a LinkedList as an example of
                                                    // collection

l.add(-8); l.add(4); l.add(0); l.add(-2);

ListIterator it = l.listIterator(); // I get the ListIterator from the list.
```

Then I can use the iterator to cycle through the elements of the List.

```
while (it.hasNext()) { // I have to check if I have another element in the collection

    Integer num = it.next(); // I get each element of the collection

    System.out.println(num);

}

// -8    4        0        -2
```

Until this point, same usage as Iterator. Now, we are going to see some added functionalities.

1. Traverse the list in the backward direction:

```
while (it.hasPrevious()) { // I have to check if I have another element in the collection

    Integer num = it.previous(); // I get each element of the collection

    System.out.println(num);

}

// -2    0        4        -8
```

2. Change some elements:

```
while (it.hasNext()) {

    int num = it.next();

    if (num < 0) it.set(num * -1);

}

System.out.println(l);
```

3. Add elements to the list. You can add elements with method `add`.
4. Get next or previous indexes. These actions are done with method `next` (checking with `hasNext()`) and with method `previous` (checking with `hasPrevious()`).